DAT471/DIT066
Computational techniques for large-scale data

Assignment 4
Deadline: 2026-05-11 23:59

Problem 1: Twitter followers (12 pts)

We will revisit last week's problem 1 [1] and reimplement it in Spark. As before, the data is found in files
`/data/courses/2026_dat471_dit066/datasets/twitter/twitter-2010_*.txt`.
Use the file
`/data/courses/2026_dat471_dit066/datasets/twitter/twiter-2010_10M.txt`
when measuring the scalability of your code. Only use RDDs in this problem; do not use any classes or functions from the `pyspark.sql` module.

(a) Implement the missing bits of `pyspark_twitter_follows.py` to determine the maximum number of people followed, the Twitter id of the account with the maximum number of people followed, the average number of people followed, and the number of accounts that follow no-one. (4 points)

(b) Measure the scalability of your algorithm on 1, 2, 4, ..., 64 cores. Plot the empirical speedup as the function of cores. In addition to the plot, report the single-core runtime on the dataset. (2 points)

(c) Implement the missing bits of `pyspark_twitter_followers.py` to determine the maximum number of followers, the Twitter id of the account with the maximum number of followers, the average number of followers, and the number of accounts that have no followers. (4 points)

(d) Measure the scalability of your algorithm on 1, 2, 4, ..., 64 cores. Plot the empirical speedup as the function of cores. In addition to the plot, report the single-core runtime on the dataset. (2 points)

Problem 2: Climate (12 pts)

The files `/data/courses/2026_dat471_dit066/datasets/climate/climate_-*.csv` contain daily measurements from various weather stations, spanning back to the 19th century. The data originates from Global Historical Climatology Network daily (GHCNd) [2] and is accessible online through NCEI [1].

The full dataset (apart from some singleton measurements) is recorded in `climate_full.csv`. The other files, labeled *tiny*, *small*, *medium*, and *large*, contain subsets of various sizes of the data, but are formatted exactly the same. That is, they are subsets of the *full* dataset.

Each row contains a daily measurement at a particular weather station somewhere around the world. The columns of the dataset are presented in Table 1.

[1] `https://www.ncei.noaa.gov/products/land-based-station/global-historical-climatology-network-daily`

Table 1: Columns of the GCHNd dataset.

Column	Explanation	Type	Unit (if applicable)
`STATION`	Alphanumeric station code	String	–
`DATE`	Date of observation	String	YYYY-MM-DD
`LATITUDE`	Latitude	Real	Degree (N positive)
`LONGITUDE`	Longitude	Real	Degree (E positive)
`ELEVATION`	Elevation from sea level	Real	m
`NAME`	Human-readable name of location	String	–
`PRCP`	Daily precipitation (rainfall)	Real	Inches
`TMAX`	Daily maximum temperature	Real	°F
`TMIN`	Daily minimum temperature	Real	°F

In this assignment, we take a (rather simplistic) look at the data and what it says about global warming: Does it look like the Earth is warming up? We do this by analyzing two kinds of aggregate values that we compute over the dataset.

First, we use simple linear regression to fit a trend line to the time series. By its nature, we would expect temperature measurements to *oscillate*: For example, in the Northen hemisphere, winters tend to be systematically colder than summers. However, over decades, if the climate were to stay stable, we would expect the trend line to have a *slope* of almost zero, and if the Earth were not warming up, we would expect there to be rather a noisy distribution of positive and negative slopes. If you don't remember how to compute simple linear regression, see Appendix A.

For the independent variable of the model (the variable x), we shall use *time*. However, dates are inconvenient to work with, so we shall convert each date to a *Julian Day Number (JDN)*. JDN is defined as the number of days since noon on Monday, January 1, 4713 BC in the proleptic Julian calendar (corresponding to 12:00 UT on November 24, 4714 BC in the proleptic Gregorian calendar). For example, the julian day number of Friday, April 4, 2025, is 2460770. This makes it easy to treat dates as continuous variables. A function for computing the JDN has been provided in the skeleton file.

For the dependent variable, the daily temperature, we use an average T_{avg} of the minimum T_{min} and maximum T_{max}, that is, $T_{\text{avg}} = \frac{T_{\text{min}}+T_{\text{max}}}{2}$.

The second aggregate value we compute is the *average temperature for the decade*, and in particular, *the difference in the decadewise averages for 1910s and 2010s*, for those stations that have records spanning both decades. That is, we map each date to a decade that is defined as the year rounded down to the nearest multiple of 10. So, YYYY-MM-DD is mapped to $10 \cdot \lfloor \frac{\text{YYYY}}{10} \rfloor$. For example, the dates 1917-12-06, 1987-01-10, and 2025-04-04 map to decades 1910, 1980, and 2020, respectively. The decadewise average for a weather station is simply the average of all T_{avg} values for that decade at that station. However, we only need to compute these values for the two decades in question.

In both cases, we record the stations with top 5 values (greatest positive slope and the greatest positive decadewise difference between 1910s and 2010s), fraction of positive instances out of all (fraction of positive slopes or positive 1910/2010 differences out of all stations), and the five-number summaries for

these values (minimum, first quartile, median, third quartile, maximum).

Table 2 shows the Top 5 slopes for the tiny, small, and medium datasets. Correspondingly, Table 3 shows the Top 5 decadewise average temperature differences for those stations for which data was found from the 1910s and 2010s; note that no such stations exist in the *tiny* dataset. Likewise, the five-number summaries for the tiny, small, and medium datasets are given in Tables 4 and 5, for the slope and the decadewise temperature difference, respectively. The slope is positive for 90.0%, 71.1%, and 69.6% of the stations for the tiny, small, and medium sets, respectively. Similarly, the decadewise temperature difference is positive for 83.3% and 77.9% of the stations, for the small and medium datasets, respectively.

Table 2: Top 5 slopes for the tiny, small, and medium datasets.

Dataset	Station	Name	Slope (°F/d)
Tiny	CA007016932	STE CATHERINE, QC CA	$1.341 \cdot 10^{-3}$
	USC00333345	GREEN, OH US	$1.053 \cdot 10^{-3}$
	USW00094732	NORTHEAST PHILADELPHIA AIRPORT, PA US	$8.829 \cdot 10^{-4}$
	JA000047770	KOBE, JA	$7.927 \cdot 10^{-4}$
	NOE00109939	TRYVASSHOGDA, NO	$7.032 \cdot 10^{-4}$
Small	MXN00005023	PALESTINA DGE, MX	$7.559 \cdot 10^{-3}$
	MXN00009052	UNIDAD MODELO, MX	$5.161 \cdot 10^{-3}$
	MXN00009003	AQUILES SERDAN 46, MX	$2.004 \cdot 10^{-3}$
	SWE00139268	FREDRIKSBERG, SW	$1.421 \cdot 10^{-3}$
	CA003052995	HAILSTONE BUTTE LO, AB CA	$1.133 \cdot 10^{-3}$
Medium	GLW00017615	CAMP CENTURY, GL	$5.372 \cdot 10^{-2}$
	UPM00033921	STEREGUSCIJ, UP	$2.498 \cdot 10^{-2}$
	CA003010232	AMISK, AB CA	$1.089 \cdot 10^{-2}$
	USC00445150	MADISON, VA US	$7.821 \cdot 10^{-3}$
	USC00417550	REMUDA, TX US	$7.655 \cdot 10^{-3}$

Table 3: Top 5 slopes for the tiny, small, and medium datasets.

Dataset	Station	Name	Temperature difference (°C)
Small	USC00313976	HENDERSONVILLE 1 NE, NC US	5.7
	USC00398472	TYNDALL, SD US	3.6
	USC00410493	BALLINGER 2 NW, TX US	1.6
	USC00401790	CLARKSVILLE WWTP, TN US	0.7
	USC00144712	LINCOLN 1 SE, KS US	0.5
Medium	USC00213455	HALLOCK, MN US	16.1
	USC00264349	LAHONTAN DAM, NV US	15.7
	RSE00151755	KON KOLODEZ, RS	13.9
	CA007051200	CAUSAPSCAL, QC CA	13.7
	USC00256290	O NEILL, NE US	13.4

Table 4: Five-number summary for the slopes. All values are given in °F/d.

Dataset	Min	Q_1	Median	Q_3	Max
Tiny	$-1.012 \cdot 10^{-3}$	$1.651 \cdot 10^{-4}$	$6.102 \cdot 10^{-4}$	$8.829 \cdot 10^{-4}$	$1.341 \cdot 10^{-3}$
Small	$-6.942 \cdot 10^{-3}$	$-2.033 \cdot 10^{-5}$	$1.514 \cdot 10^{-4}$	$6.698 \cdot 10^{-4}$	$7.559 \cdot 10^{-3}$
Medium	$-2.564 \cdot 10^{-1}$	$-9.197 \cdot 10^{-5}$	$2.982 \cdot 10^{-4}$	$7.291 \cdot 10^{-4}$	$5.372 \cdot 10^{-2}$

Table 5: Five-number summary for the decadewise temperature differences. All values are given in °C.

Dataset	Min	Q_1	Median	Q_3	Max
Small	−2.7	0.5	0.7	3.6	5.7
Medium	−23.6	0.3	5.3	8.6	16.1

(a) Using `pyspark_climate.py` as your starting point, implement a Spark program that, using `pyspark.sql` Data Frames, computes the values requested, that is,

1. Computes the Julian date number for each record,
2. Computes $T_{\text{avg}} = \frac{T_{\text{min}}+T_{\text{max}}}{2}$,
3. For each station, performs fits a simple linear regression model using least squares (T_{avg} as function of JDN),
4. Prints the station code, station name, and the slope of the top 5 stations with the greatest slope,
5. Prints the five-number summary of the slopes,
6. Prints the fraction of positive slopes,
7. Computes the difference in decadewise averages between 2010s and 1910s for those stations that have measurements from both the 1910s and 2010s,
8. Prints the station code, station name, and the decadewise temperature difference of the top 5 stations with the greatest positive difference
9. Prints the five-number summary of the temperature differences, and
10. Prints the fraction of positive temperature differences.

(6 pts)

(b) Determine the scalability of your solution. Use the *large* dataset for your measurements. Plot the speedup as the function of CPU cores for $w = 1, 2, 4, \ldots, 64$ workers. (2 pts)

(c) Determine a breakdown of the time: how large fraction of the time is spent in reading the data, and how much in computations. What is the speedup you get for *computations only* using 64 cores? (1 pt)

(d) Compute the aforementioned values for the full dataset. Record the station codes, names, and values for the top 5 slopes and temperature differences as nicely formatted tables in your report. Also report the five-number summaries as nicely formatted tables. Record the fraction of positive values out of all values computed. Also record the total wall-clock running time of your code and how many workers you used. (2 pts)

(e) The data has not been sanitized and some of the measurements (particulary very old ones) are likely erroneous, which may lead to some non-sensical values. Still, the five-number summaries should give us a hint at the big picture. What are your thoughts: what does the data tell us about climate change? (1 pt)

Hints

- Obviously, for Problem 1, you should get the same results as with the code you used before. This is a good *lithmus test* for the correctness of your solution.
- Using `cache()` correctly can have a drastic effect on performance.
- If you are new to Spark, keep the cheat sheet at hand.
- The `flatMap()` function is very useful if you need to emit multiple values.
- Note that Spark requires Java 8, 11, or 17. The default version on your system will not most likely work. Here is a suitable invocation for creating a PySpark environment if you want to create the environment yourself:
  `$ conda create -n pyspark -c conda-forge pyspark openjdk\<18`
- As in previous assignments, you can use a pre-made container located in `/data/courses/2026_dat471_dit066/containers/assignment4.sif`.
- In Problem 2, some useful functions include `select()`, `withColumn()`, `applyInPandas()`, and `approxQuantile()`.

Returning your assignment

Return your assignment on Canvas. Your submission should consist of a report that answers all questions as PDF file (preferably typeset in LaTeX) called `assignment4.pdf`. In addition, you should provide the code you used in Problems 1a, 1c, and 2 as `assignment4_problem1a.py`, `assignment4_problem1c.py` and `assignment4_problem2.py`, respectively, stored inside a file called `assignment4.zip`. The code for Problems 1a, 1c, and 2 must match the interfaces of `pyspark_twitter_follows.py`, `pyspark_twitter_followers.py`, and `pyspark_climate.py`; the command line parameters must not be changed, and output must be correct. Do *not* deviate from the requested filenames and do *not* produce the plots in these files; these files will be used for evaluating the quality of your implementations automatically.

References

[1] Haewoon Kwak et al. "What is Twitter, a Social Network or a News Media?" In: *Proceedings of the 19th international conference on World wide web (WWW '10)*. 2010. DOI: https://doi.org/10.1145/1772690.1772751.

[2] Matthew J. Menne et al. "An Overview of the Global Historical Climatology Network-Daily Database". In: *Journal of Atmospheric and Oceanic Technology* (2012), pp. 897–910. DOI: https://doi.org/10.1175/JTECH-D-11-00103.1.

A Simple linear regresssion

Suppose we are given n pairs of observations $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$. We want to fit a model $y = \alpha + \beta x$, such that the sum of squared residuals $\sum_{i=1}^{n} \varepsilon^2 = \sum_{i=1}^{n} (y_i - \alpha - \beta x_i)^2$ is minimized. Denote the means by $\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^{n} y_i$. Write $\Delta x_i = x_i - \bar{x}$, $\Delta x_i^2 = (x_i - \bar{x})^2$, and $\Delta y_i = y_i - \bar{y}$. The least squares estimate for the parameters α and β are

$$\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x}\,, \text{ and}$$
$$\hat{\beta} = \frac{\sum_{i=1}^{n} \Delta x_i \Delta y_i}{\sum_{i=1}^{n} \Delta x_i^2}\,.$$